Docker Learning Series – Day 5 (Part 1)

Docker Networking Fundamentals

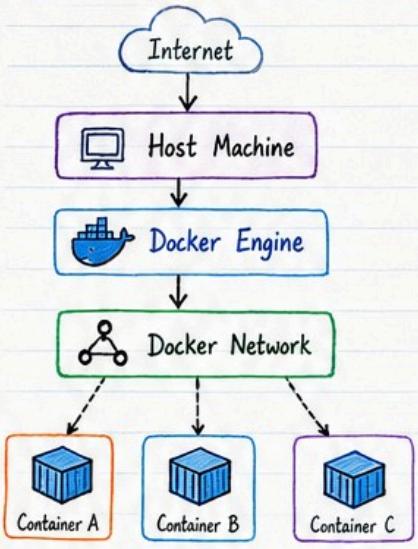
Container Communication • Docker Networks • Bridge Network

Why Docker Networking?

- Why containers need networking
- Communication between containers
- External access to containers
- Network isolation
- Security

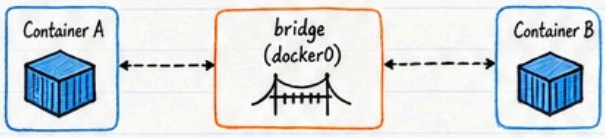
> Networking is the backbone of containerized applications!

2 Docker Network Architecture



- **Docker Daemon** Manages containers, images, networks and volumes.
- **Virtual Ethernet (veth)** Creates virtual network interfaces for each container.
- **Linux Bridge** Connects multiple containers together on the same network.
- **Network Namespace** Provides isolated network environment for each container.

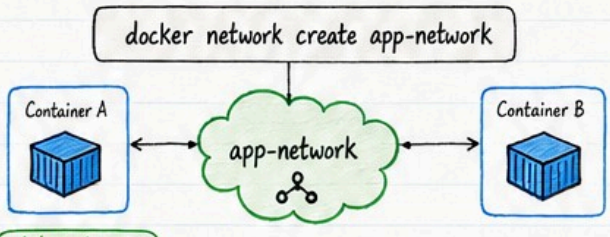
3 Default Bridge Network



- Default network created by Docker.
- All containers connect to bridge network automatically if no network is specified.
- Containers can communicate with each other on this network.
- Containers are isolated from host and other external networks by default.
- Uses NAT for external access.

> Default Network
> Name: bridge
> Driver: bridge
> Scope: local

4 User-defined Bridge Network



Advantages

- Automatic DNS resolution
- Better isolation
- Easier container communication using names
- Customizable and production friendly

Best Practice!

5 Useful Commands

`$ docker network ls`	# List all networks
`$ docker network inspect bridge`	# Inspect bridge network
`$ docker network create app-network`	# Create user-defined network
`$ docker network rm app-network`	# Remove network
`$ docker run --network app-network nginx`	# Run container in specific network
`$ docker network connect app-network <container_id>`	# Connect existing container to network
`$ docker network disconnect app-network <container_id>`	# Disconnect container from network

6 Production Notes

- Use user-defined bridge network in production.
- Avoid relying on container IPs, they can change.
- Use container names for communication.
- Inspect networks regularly.
- Always follow least privilege networking.

7 Interview Questions

- Q1: What is Docker networking?
- Q2: What is the difference between Default Bridge network and User-defined Bridge network?
- Q3: How do containers communicate with each other in Docker?

Quick Tip

Container networking may look complex, but it's built on simple Linux networking concepts!

Key Takeaway

Docker networking enables containers to communicate securely, efficiently and reliably. Understanding bridge networks is the first step to mastering advanced Docker Networking!

Docker makes development faster, deployment easier and operations smarter!

Keep Learning. Keep Building. Keep Growing!
— DevOps Engineer

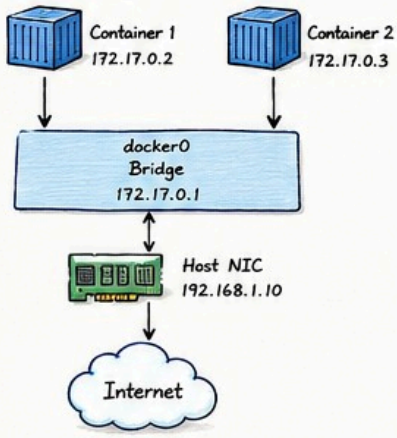
Docker Learning Series – Day 5 (Part 2)

Docker Network Drivers

Bridge • Host • None • Overlay • Macvlan

Bridge Network (Default)

Architecture



Use Cases

- Default network
- Single host
- Web applications
- Microservices

Advantages

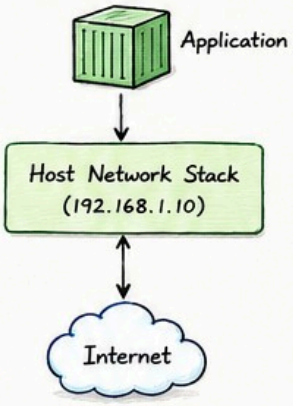
- ✓ Isolation between containers
- ✓ Easy to use
- ✓ Good for single host setups

Limitations

- × Not for multi-host communication
- × NAT can impact performance

Host Network

Architecture



Explain

- No isolation
- High performance
- Shares host network directly
- Containers use host's IP and ports

Command

```
docker run
--network host
nginx
```

None Network

Architecture

Container
(No Network)

Explain

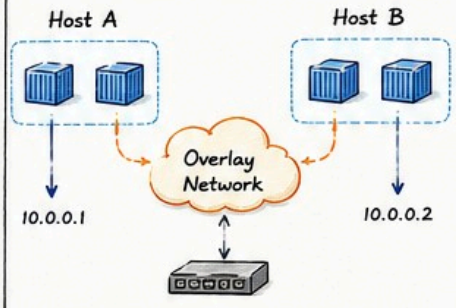
- No IP Address
- No Internet Access
- Complete isolation
- Used for highly secure environments

Command

```
docker run
--network none
alpine
```

Overlay Network

Architecture

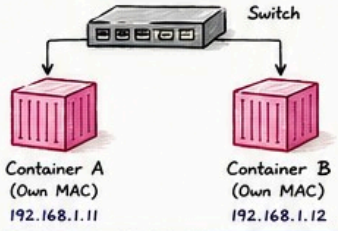


Explain

- Enables multi-host communication
- Used in Docker Swarm
- Creates a virtual network across multiple hosts
- Ideal for distributed applications

Macvlan Network

Architecture



Explain

- Each container gets its own MAC Address
- Appears as physical machine on the network
- Useful for legacy applications
- Directly connected to physical network

Note

Choose the right network driver based on your application requirements!

Useful Commands

```
# List all networks
docker network ls

# Inspect a network
docker network inspect <name>

# Create a network
docker network create \
--driver <driver_name> <network_name>

# Example: Create overlay network
docker network create \
--driver overlay my-overlay
```

Docker Network Drivers – Comparison

Driver	Isolation	Performance	Multi-host	Internet Access	Production Use
Bridge (Default)	High	Good	No	Yes (via NAT)	Yes (Single Host)
Host	None	Excellent	No	Yes	Yes (High Performance)
None	Complete	N/A	No	No	Rare (Special Use)
Overlay	High	Good	Yes	Yes	Yes (Swarm / Multi-host)
Macvlan	Low (L2 Level)	Excellent	Yes*	Yes	Yes (Legacy / Advanced)

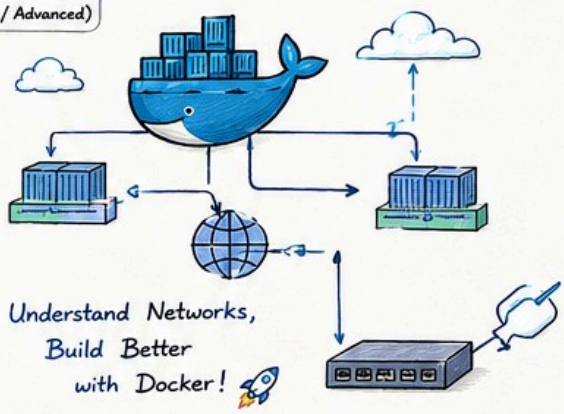
* Macvlan supports multi-host with additional network configuration.

Interview Questions

1. Which network driver is the default in Docker?
   > Bridge network is the default driver.
2. What is the difference between Overlay and Bridge?
   > Bridge works on a single host, while Overlay enables communication between containers across multiple hosts.
3. What is the difference between Host and Bridge?
   > Host shares the host's network stack (no isolation), while Bridge provides isolation using NAT.
4. When should you use Macvlan?
   > When containers need their own MAC address and should appear as physical machines on the network (e.g., legacy applications).

Key Takeaways

- ✓ Docker provides multiple network drivers to fit different use cases.
- ✓ Bridge is default and great for most scenarios.
- ✓ Overlay is best for multi-host container communication.
- ✓ Host gives maximum performance but no isolation.
- ✓ Choose the right driver for security, performance and scalability!



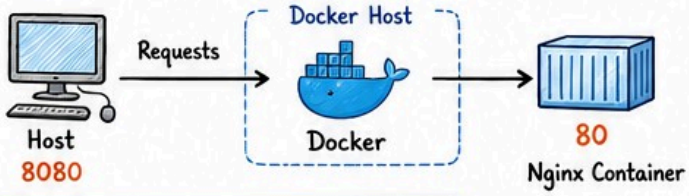
Understand Networks,
Build Better
with Docker!

Docker Learning Series – Day 5 (Part 3)

Port Mapping • Docker DNS • Container Communication

DevOps Journey

Port Mapping



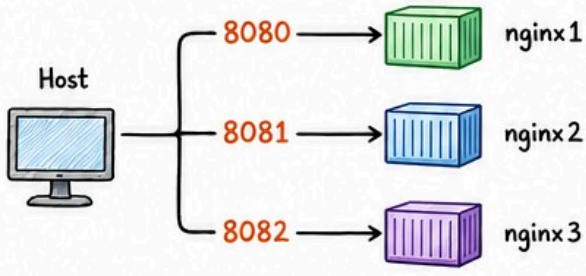
Command:

```
docker run -d -p 8080:80 nginx
```

- Host Port (8080) → Port on your machine
- Container Port (80) → Port inside the container
- NAT (Network Address Translation) is used by Docker to map Host Port to Container Port
- Allows external access to the application running inside the container

External Access

Multiple Containers – Multiple Ports




```
docker run -d -p 8080:80 --name nginx1 nginx
docker run -d -p 8081:80 --name nginx2 nginx
docker run -d -p 8082:80 --name nginx3 nginx
```

Expose vs Publish

In Dockerfile

```
FROM nginx
EXPOSE 80
```

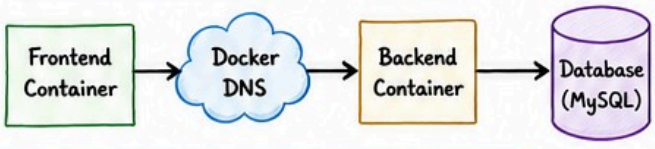
VS

At Runtime

```
docker run -d
-p 8080:80 nginx
```

Aspect	EXPOSE	-p / --publish
Purpose	Informs Docker that the container listens on a specific port	Publishes the port to the host for external access
Accessibility	Internal only (within Docker network)	External (outside world can access)
Mandatory	Not mandatory	Required for external access
Used in Production	No ✗	Yes ✓
Example	EXPOSE 80	docker run -p 8080:80 nginx

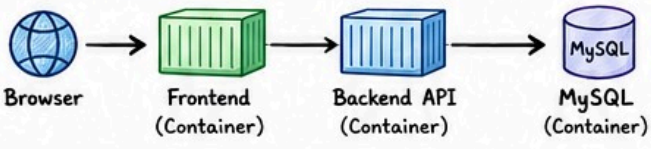
Docker DNS



- Docker has an embedded DNS server (127.0.0.11)
- Containers resolve each other by container name
- Works automatically in user-defined bridge networks
- No need to hardcode IP addresses
- Makes applications portable & scalable

Container Name Resolution

Container Communication



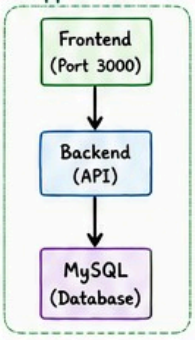

```
# Connect to frontend and ping backend
docker exec -it frontend ping backend

# Connect to backend and ping db
docker exec -it backend ping db
```

Containers communicate over the Docker Network

Real Example (End-To-End App)

1. `docker network create app-network`
2. `docker run -d --name db \ --network app-network mysql`
3. `docker run -d --name backend \ --network app-network backend`
4. `docker run -d --name frontend \ --network app-network -p 3000:3000 frontend`

app-network



Production Tips

- ☑ Never hardcode container IPs
- ☑ Use Docker DNS (container names)
- ☑ Publish only required ports
- ☑ Keep database private (no direct access from host)
- ☑ Use user-defined networks for better isolation
- ☑ Follow least privilege principle
- ☑ Monitor & secure your containers

Secure • Scalable • Reliable

Interview Questions

- ★ Q1. What is the difference between EXPOSE and -p?
- ★ Q2. How does Docker DNS work?
- ★ Q3. How do containers communicate with each other?
- ★ Q4. What is the default DNS server IP in Docker?
- ★ Q5. How would you make a container accessible from the outside world?

Think Like a DevOps Engineer!

REMEMBER:
Docker makes networking simple, but understanding it deeply makes you a better DevOps Engineer!

CONTAINERS · NETWORKING · DNS · COMMUNICATION

Keep Learning
Keep Growing
Stay DevOps

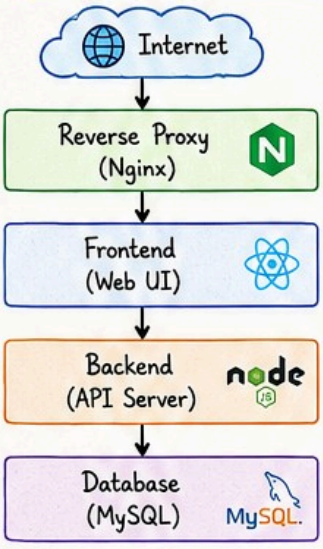
Docker Learning Series – Day 5 (Part 4)

Production Networking & Best Practices

Security • Isolation • Troubleshooting

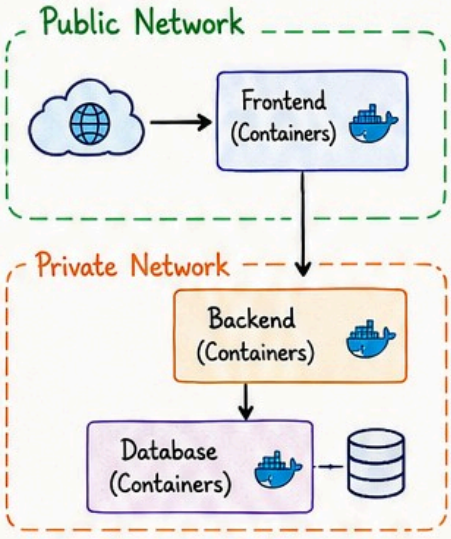
Production Ready Docker Networking

1 Three-Tier Architecture



- Internet traffic enters via Reverse Proxy
- Frontend talks to Backend
- Backend communicates with Database
- Each layer runs in its own container(s)

2 Network Isolation



- Public vs Private Networks
- Security Boundaries
- Least Exposure
- Only necessary ports exposed to public
- Internal components stay private

Keep your database and backend in private networks. Expose only what is necessary to the public.

3 Best Practices

- **Use User-Defined Bridge Networks** — Better isolation and DNS resolution.
- **Use Container Names, Not IPs** — Docker DNS handles container discovery.
- **Expose Only Necessary Ports** — Follow the principle of least exposure.
- **Separate Frontend & Backend Networks** — Avoid direct access to backend.
- **Keep Database Internal** — Never expose database ports.
- **Principle of Least Privilege** — Run containers with least required access.

4 Troubleshooting Commands

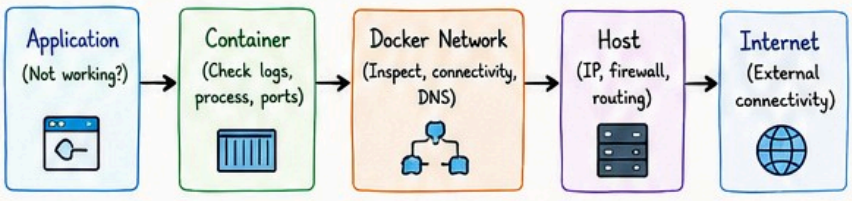
`docker network ls` — List all networks	`docker logs <container>` — Check container logs
`docker network inspect <network>` — Inspect network details	`ip addr` — Check IP address
`docker inspect <container>` — Inspect container details	`ping <hostname / IP>` — Test connectivity
`docker port <container>` — Check port mappings	`curl <url / IP>` — Test HTTP connectivity
`docker exec -it <container> sh` — Enter container shell	

Use these commands to diagnose networking issues quickly!

5 Common Problems

- **Container cannot reach another container** — Wrong network or network not connected.
- **DNS resolution failed** — Container name not resolvable.
- **Port already allocated** — Host port already in use.
- **Wrong network** — Container is not in the expected network.
- **Firewall issues** — Host firewall blocking traffic.

6 Debugging Flow



Follow the flow step-by-step to identify where the issue is occurring.

7 Interview Questions

- How do you troubleshoot Docker networking issues?
- How do you isolate containers in Docker?
- How do you secure production Docker networks?

Key Takeaway

A strong Docker networking setup ensures security, scalability, and reliability in production environments.

Good Networking = Secure, Scalable & Reliable Applications

Stay Secure Stay Updated!